

PROJECT AND TEAM INFORMATION

Project Title

Inverted Index and Code-Based Search Engine

Student / Team Information

Team Name: Team #	Code Acer DSCPP-III-2026-T112
Team member 1 (Team Lead) <i>(Last Name, name: student ID: email, picture):</i>	Gupta, Anmol – 2510385219 anmolgupta1523@gmail.com 
Team member 2 <i>(Last Name, name: student ID: email, picture):</i>	Kumar, Harshit – 2510012451 harshitkumarlearning@gmail.com 
Team member 3 <i>(Last Name, name: student ID: email, picture):</i>	Goyal, Atharva – 2510360047 atharvagoyal45@gmail.com 
Team member 4 <i>(Last Name, name: student ID: email, picture):</i>	Nainwal, Navneet – 2510010627 nainwalnavneet@gmail.com 

PROPOSAL DESCRIPTION

Motivation

Searching through a huge database for only a single code line or document takes a lot of time because of traditional searching techniques which read documents in step by step manner. In particular cases like code repositories, searching involves developers spending time in searching for functions or variables which could delay the process.

The method of Inverted Index and Code Search Engine changes the way we look for text and the way codes are searched. The system creates an index that connects word and the file path. Whenever someone does a search the engine looks at the index of reading every single document. This project uses the data structures and algorithms. The hash tables is used for the index process and key word detection. Linked lists is used to keep track of references.

The process is handled by trees along with sorting algorithms. While processing, the engine is able to process that programming structures such as keywords and variables are not words. It will provide the sorted results using data

State of the Art/Current solution

At present, state-of-the-art search engines such as Google, Elasticsearch, and Apache Lucene are applying innovative approaches in order to able to conduct fast and efficient searches. The main techniques is the application of inverted indexes, which allow storing keywords along with their documents. Such an approach provides the possibility to find the wanted results without searching the whole file for every query.

Before a document can be indexed the text must be divided into parts. After that the text is normalized and the text stop words has removed. In some of the cases the text may need stemming. These steps helps to improvise the quality of the retrieval process. Search engines rely upon several ranking techniques such as TF-IDF and BM25 to determine how relevant the results are. While searching the query is checked against the index instead of looking through every document, which makes it faster and more efficient.

For source-code searching, platforms like GitHub and advanced IDEs have options for searching keywords, functions, classes, and variables throughout code files. Though such systems are quite capable and available for large-scale projects, we find them internally quite complicated.

The initiative introduced in our project is the creation of a simplified code search engine from the beginning. It concentrates on the basic principles of contemporary search engines through the employment of methods such as inverted indexing, hashing, searching, sorting, document retrieval employing DSA techniques.

Project Goals and Milestones

Goal:

The development of a swift and uncomplicated search engine based on coding principles that utilizes an inverted index in searching text for any particular keywords found in numerous coding files.

Key early achievements:

- 1. Setting up an inverted index for the keywords and files.*
- 2. Using of hashing and searching algorithms for carrying out fast searching.*
- 3. Look at source-code files such as C++, Java, and Python.*
- 4. Display the following files that match and the locations of the code.*
- 5. Apply DSA concepts such as hashing, linked lists, trees, sorting, and searching.*

Milestones:

- 1. Analyze requirements and design the system.*
- 2. Read and process source-code files.*
- 3. Build the inverted index.*
- 4. Implement the search and sorting modules.*
- 5. Develop a basic user interface.*
- 6. Test and optimize the system.*

Project Approach

This project is developed by first collecting and processing multiple source-code files. Necessary keywords will be obtained by the system and an inverted index will be formed, which associates the keywords with the files as well as their locations.

The search module will use the index to quickly find matching keywords without scanning every file. Hashing, linked lists, trees, searching, and sorting algorithms will be used to improve efficiency and organize the results.

A normalized user interface will allow users to enter a keyword and view the relevant files and code locations. The system will be tested using different source-code files and optimized for faster searching.

Technologies & Platforms:

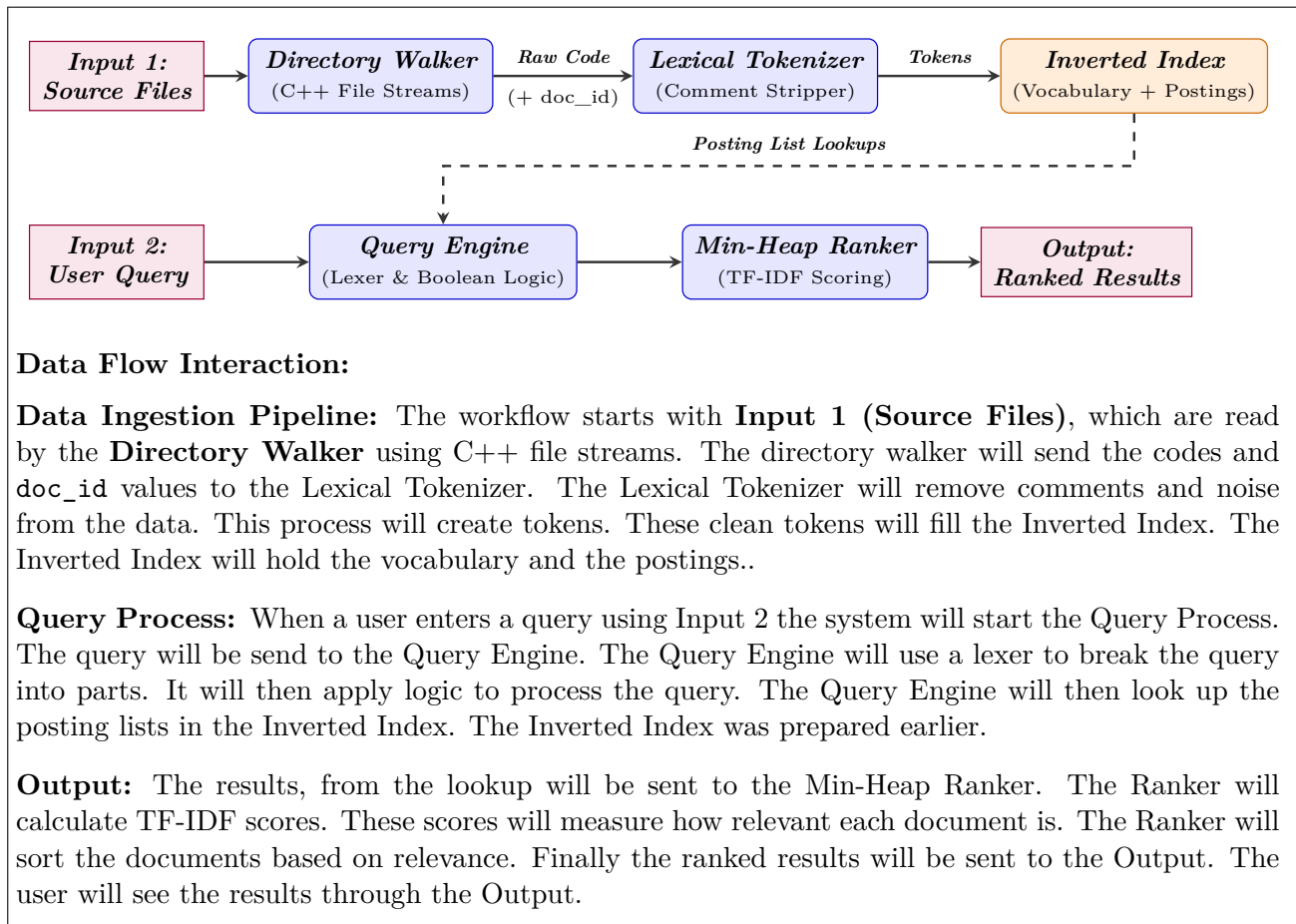
C++ – Core logic and DSA implementation

HTML, CSS, JavaScript – Basic user interface

VS Code – Development environment

Windows/Linux – Execution and testing platform

System Architecture (High Level Diagram)



Project Outcome/Deliverables

The project will deliver a standalone, high-performance CLI-based search engine written in C/C++ for indexing and searching local source-code repositories.

Key deliverables:

- A Word Inverted Index with dynamic posting lists for fast keyword lookup.
- Efficient memory management using `malloc()` and `free()`.
- Retrieving multiple keywords faster without repeating linear file scanning.
- Supported Boolean queries using posting list operation.
- Using a scoring mechanism and priority queue for ranking of matching files of TOP – K relevance.
- The binary index persistence to retrieve the indexed data means avoiding reindexing.
- The system supports searching for keywords, functions, variables and classes.
- It adds source code, documentation and test cases for evaluating the search efficiency and accuracy.

The project shows how data structure and algorithm concepts, like hashing, linked lists, pointers, recursion, sorting, searching and priority queues are used in practice to build a real-world code search engine.

Assumptions

- *Source File Format:* The engine assumes all indexed targets are text-readable source code files (such as .c, .cpp, .h, .java, .py, .txt), ignoring binary, compiled, or executable formats (.o, .exe, .bin).
- *Character Encoding and Identifiers:* Code identifiers, keywords, and search queries are assumed to use standard ASCII alphanumeric characters and underscores(_), with non-alphanumeric symbols treated as token delimiters.
- *System Memory Capacity:* The combined vocabulary of unique keywords and associated posting lists is assumed to fit entirely within system RAM during runtime.
- *File Stability During Ingestion:* Target source files are assumed to remain static and unmodified on disk while the engine is actively reading, tokenizing, and constructing the in-memory inverted index.
- *Case-Insensitive Ingestion:* All tokens and search queries are assumed to be normalized to lower-case for uniform indexing and exact retrieval.

References

- *Introduction to Information Retrieval* – Manning, Raghavan, Schütze (Cambridge UP) (<https://nlp.stanford.edu/IR-book/>)
- *Mastering Algorithms with C* Kyle Loudon (O'Reilly Media).
- *The C++ Programming Language* – Bjarne Stroustrup.
- *Lucene/Elasticsearch Inverted Index Architecture & TF-IDF Retrieval Models.* (<https://www.elastic.co/guide/en/elasticsearch/reference/current/documents-indices.html>)